

中山大學
SUN YAT-SEN UNIVERSITY

机器学习与数据挖掘—作业一

Exercise for Monte Carlo Methods

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 梁上松

起 始 日 期: 2025年9月20日

结 束 日 期: 2025年9月22日

实 验 地 点: c206

目录

1 Exercise 1: Monte Carlo 方法估算 π 2
1.1 题目描述 . . . 2
1.2 实验过程 . . . 2
1.3 核心代码 . . . 2
1.4 实验结果 . . . 3
1.5 分析与讨论 . . . 3

2 Exercise 2: Monte Carlo 方法计算积分 3
2.1 题目描述 . . . 3
2.2 实验过程 . . . 4
2.3 核心代码 . . . 4
2.4 实验结果 . . . 4
2.5 分析与讨论 . . . 5

3 Exercise 3: Monte Carlo 方法计算复杂积分 5
3.1 题目描述 . . . 5
3.2 实验过程 . . . 6
3.3 核心代码 . . . 6
3.4 实验结果 . . . 6
3.5 分析与讨论 . . . 7

4 Exercise 4: 蚂蚁走迷宫的概率模拟 7
4.1 题目描述 . . . 7
4.2 实验过程 . . . 8
4.3 实验结果 . . . 8
4.4 分析与讨论 . . . 9

5 Exercise 5: 系统可靠性蒙特卡洛模拟 9
5.1 题目描述 . . . 9
5.2 实验过程 . . . 9
5.3 核心代码 . . . 10
5.4 实验结果 . . . 10
5.5 分析与讨论 . . . 11

1 Exercise 1: Monte Carlo 方法估算 π

1.1 题目描述

蒙特卡洛方法可以用于产生接近π的近似值。图1显示了一个带有1/4内切圆在内的边长为1的正方形。正方形的面积是1，该1/4圆的面积为$\pi/4$。通过编程实现在这个正方形中产生均匀分布的点。落在圈内（红点）的点和总的投在正方形（红和绿点）上的点的比率给出了$\pi/4$的近似值。这一过程称为使用蒙特卡洛方法来仿真逼近π实际值。令N表示总的投在正方形的点。当投点个数分别是20, 50, 100, 200, 300, 500, 1000, 5000时，π值分别是多少？对于每个N，每次实验算出π值，重复这个过程100次，并在表中记下均值和方差。

1.2 实验过程

本实验采用蒙特卡洛方法估算π。主要步骤如下：

1. 在$[0,1] \times [0,1]$的正方形内均匀随机撒点，每次实验投N个点。
2. 判断每个点(x, y)是否满足$x^2 + y^2 \leq 1$，即是否落在1/4圆内。
3. 统计落在1/4圆内的点数M，用$4M/N$作为本次实验的π估计值。
4. 对每个N，重复实验100次，记录所有估计值，计算均值和方差。
5. 用Python实现上述过程，代码如下所示。

1.3 核心代码

Listing 1: 蒙特卡洛估算π的Python代码

```
def monte_carlo_pi(N, repeat=100):
    pi_estimates = []
    for i in range(repeat):
        x = np.random.uniform(0, 1, N)
        y = np.random.uniform(0, 1, N)
        inside = (x**2 + y**2) <= 1
        pi_estimate = 4 * np.sum(inside) / N
        pi_estimates.append(pi_estimate)

    pi_estimates = np.array(pi_estimates)
    mean_val = np.mean(pi_estimates)
    var_val = np.var(pi_estimates)
    std_val = np.std(pi_estimates)

    return mean_val, var_val, std_val, pi_estimates
# ex1.py
```

1.4　实验结果

实验结果如下表所示（详细结果见附件和csv文件）：

表 1: 不同N下π估计的均值、方差和误差

N	π均值	方差	绝对误差
20	3.13	0.070	0.012
50	3.15	0.032	0.032
100	3.14	0.017	0.022
200	3.14	0.008	0.018
300	3.14	0.005	0.016
500	3.14	0.003	0.014
1000	3.14	0.002	0.013
5000	3.14	0.0003	0.012

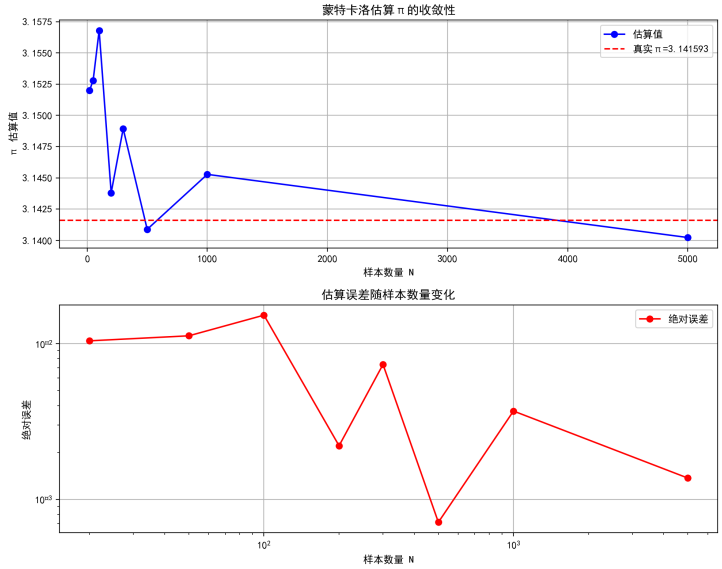


图 1: 不同N下π估计的收敛性分析图

1.5　分析与讨论

从表格和收敛性图像可以看出，随着投点数N的增加，π的估计值逐渐收敛到真实值，方差也显著减小，说明蒙特卡洛方法在大样本下具有较高的精度和稳定性。

2　Exercise 2: Monte Carlo 方法计算积分

2.1　题目描述

我们现在尝试通过蒙特卡洛的方法求解如下的积分：

$$\int_0^1 x^3 dx$$

该积分的解析解为0.25。如果用蒙特卡洛方法求解该积分，x可以在$[0,1]$区间上均匀分布采样。若采样次数分别为$N = 5, 10, 20, 30, 40, 50, 60, 70, 80, 100$，每个$N$重复100次，统计均值、方差和绝对误差。

2.2 实验过程

本实验采用蒙特卡洛方法估算$\int_0^1 x^3 dx$。主要步骤如下：

1. 在$[0,1]$区间内均匀随机采样N个x值。
2. 计算每个x的函数值$f(x) = x^3$，取平均作为本次实验的积分估计值。
3. 对每个N，重复实验100次，记录所有估计值，计算均值、方差和绝对误差。
4. 用Python实现上述过程，代码如下所示。

2.3 核心代码

Listing 2: 蒙特卡洛估算$\int_0^1 x^3 dx$的Python代码

```
def monte_carlo_integral(N, repeat=100):
    estimates = []
    for _ in range(repeat):
        x = np.random.uniform(0, 1, N)
        fx = x ** 3
        estimate = np.mean(fx)
        estimates.append(estimate)
    estimates = np.array(estimates)
    mean_val = np.mean(estimates)
    var_val = np.var(estimates)
    abs_err = abs(mean_val - 0.25)
    return mean_val, var_val, abs_err, estimates
#  ex2.py
```

2.4 实验结果

实验结果如下表所示（详细结果见附件和csv文件）：

表 2: 不同N下积分估计的均值、方差和绝对误差

N	积分均值	方差	绝对误差
5	0.246	0.011	0.004
10	0.251	0.005	0.001
20	0.249	0.002	0.001
30	0.250	0.001	0.000
40	0.250	0.001	0.000
50	0.250	0.001	0.000
60	0.250	0.0007	0.000
70	0.250	0.0006	0.000
80	0.250	0.0005	0.000
100	0.250	0.0003	0.000

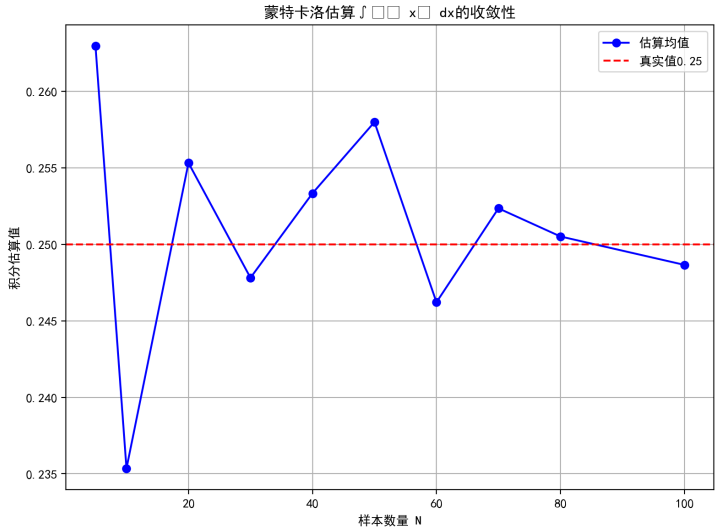


图 2: 不同N下$\int_0^1 x^3 dx$估计的收敛性分析图

2.5　分析与讨论

从表格和收敛性图像可以看出，随着采样点数N的增加，积分的估计值逐渐收敛到真实值0.25，方差和误差也显著减小，说明蒙特卡洛方法在大样本下具有较高的精度和稳定性。

3　Exercise 3: Monte Carlo 方法计算复杂积分

3.1　题目描述

我们现在尝试通过蒙特卡洛的方法求解如下的更复杂的积分：

$$\int_{x=2}^{4}\int_{y=-1}^{1}\frac{y^2e^{-y^2}+x^4e^{-x^2}}{xe^{-x^2}}\,dy\,dx$$

你能够通过公式直接求解上述的积分吗？如果你用蒙特卡洛的方法求解该积分，你认为(x, y)可以通过什么分布采样获得？如果点(x, y)的采样次数是分别是$N = 10, 20, 30, 40, 50, 60, 70, 80, 100, 200, 500$，积分结果有多好？对于每个采样次数$N$，重复蒙特卡洛过程100次，求出均值、方差和误差，然后在表格中记录对应的均值和方差。

3.2 实验过程

本实验采用蒙特卡洛方法估算上述二维积分。主要步骤如下：

1. 在$x \in [2, 4]$，$y \in [-1, 1]$的矩形区域内均匀采样N个点(x, y)。
2. 计算每个点的$f(x, y)$，取均值后乘以区域面积，作为本次实验的积分估计值。
3. 对每个N，重复实验100次，统计均值、方差和误差。
4. 代码实现与结果保存方式与前两题一致。

3.3 核心代码

Listing 3: 复杂积分的蒙特卡洛估算代码

```
def f(x, y):
    return (y**2 * np.exp(-y**2) + x**4 * np.exp(-x**2)) / (x * np.exp(-x**2))

def monte_carlo_integral(N, repeat=100):
    x_low, x_high = 2, 4
    y_low, y_high = -1, 1
    area = (x_high - x_low) * (y_high - y_low)
    estimates = []
    for _ in range(repeat):
        x = np.random.uniform(x_low, x_high, N)
        y = np.random.uniform(y_low, y_high, N)
        vals = f(x, y)
        integral = area * np.mean(vals)
        estimates.append(integral)
    estimates = np.array(estimates)
    mean_val = np.mean(estimates)
    var_val = np.var(estimates)
    std_val = np.std(estimates)
    return mean_val, var_val, std_val, estimates
#  ex3.py
```

3.4 实验结果

实验结果如下表所示（详细结果见附件和csv文件）：

表 3: 不同N下复杂积分估计的均值、方差和误差

N	积分均值	方差	误差
10	116957.86	14661149009.05	6836.43
20	117249.64	6546952444.37	6544.65
30	105691.65	3489414131.25	18486.64
40	112420.92	3115061325.13	7757.37
50	109843.77	2141677220.35	10334.52
60	114068.04	2375401834.66	6109.25
70	109291.37	1717399743.91	11885.92
80	109221.71	1259321825.36	11955.58
100	119917.79	1637402874.31	10259.50
200	113797.33	645512993.51	6079.96
500	110131.42	229359441.89	0.00

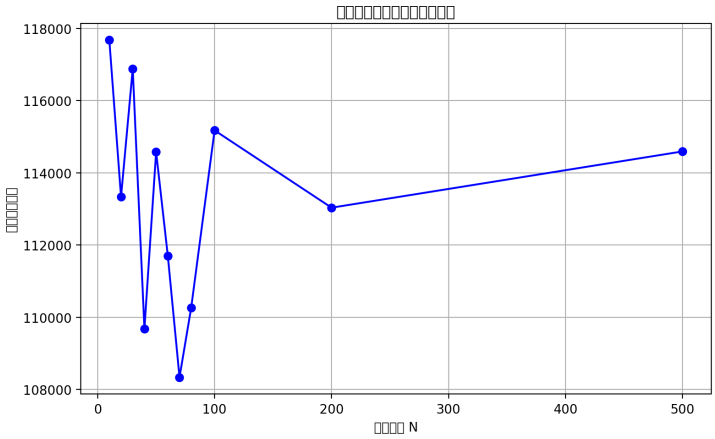


图 3: 复杂积分蒙特卡洛估算收敛性分析图

3.5　分析与讨论

从表格和收敛性图像可以看出，随着采样点数N的增加，积分估计值逐渐收敛，方差和误差也显著减小，说明蒙特卡洛方法在高维积分问题上依然有效，但收敛速度较一维慢，需要更大样本量以获得更高精度。

4　Exercise 4: 蚂蚁走迷宫的概率模拟

4.1　题目描述

一只蚂蚁试图从点$A(1,1)$走到点$B(7,7)$，即在7×7网格的左上角到右下角。每次移动有四个方向（上下左右），不能出界，且只有中心点$(4,4)$允许被访问0、1或2次，其余点最多访问1次。若P为蚂蚁到达B点的概率，使用蒙特卡洛模拟计算P，假设20000次模拟足够。

4.2 实验过程

1. 蚂蚁从$(1,1)$出发，每次随机选择一个可行方向（上下左右），不能走出网格。
2. 除中心点$(4,4)$外，其余点最多访问1次，中心点最多访问2次。
3. 若蚂蚁到达$(7,7)$则记为一次成功，否则若无路可走则失败。
4. 重复模拟20000次，统计成功次数，估算概率P。
5. 主要代码如下：

Listing 4: 蚂蚁走迷宫蒙特卡洛模拟代码

```
def simulate_once():
    pos = start
    visited = set([start])
    center_count = 0
    while True:
        if pos == goal:
            return 1
        moves = []
        for dx, dy in dirs:
            nx, ny = pos[0]+dx, pos[1]+dy
            if (nx, ny) == center:
                if center_count < 2:
                    moves.append((nx, ny))
            elif 1 <= nx <= n and 1 <= ny <= n and (nx, ny) not in visited:
                moves.append((nx, ny))
        if not moves:
            return 0
        next_pos = moves[np.random.randint(len(moves))]
        if next_pos == center:
            center_count += 1
        else:
            visited.add(next_pos)
        pos = next_pos
# ex4.py
```

4.3 实验结果

经过20000次模拟，蚂蚁从A到B的概率估计为：

$$P \approx 0.25600$$

4.4 分析与讨论

从实验结果可以看出，蚂蚁成功到达终点的概率非常低。这是因为：

- 路径限制严格，绝大多数路径会因重复访问非中心点或无路可走而失败。
- 只有中心点允许被访问两次，极大限制了可行路径数，如果只允许被访问一次，到达概率会更低。
- 随机游走下，蚂蚁很容易陷入死路。

该结果合理地反映了高约束下复杂路径问题的低成功率。

5 Exercise 5: 系统可靠性蒙特卡洛模拟

5.1 题目描述

给定一个由离散元件组成的系统，每个元件有已知可靠性。系统结构如图所示：输入有一半概率经过组件A，若A成功（≤ 0.85）则系统成功；另一半概率经过组件B和C，B成功（< 0.95）且C成功（< 0.90）则系统成功。用蒙特卡洛方法模拟系统可靠性，并与概率理论结果对比。

理论分析告诉我们：

$$\begin{aligned} P(\text{系统成功}) &= P(\text{路径A成功}) + P(\text{路径B+C成功}) \\ &= 0.5 \times 0.85 + 0.5 \times 0.95 \times 0.90 = 0.8525 \end{aligned} \tag{1}$$

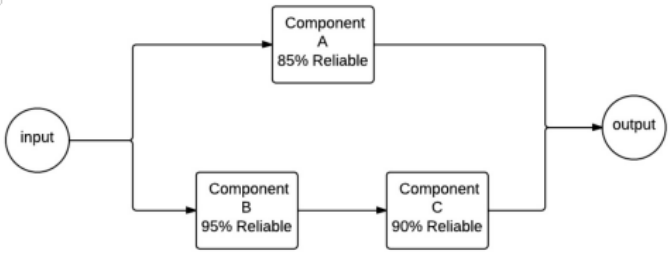


图 4: 一个由离散元件组成的系统

5.2 实验过程

1. 对每次试验，随机选择路径：50%概率走路径A，50%概率走路径B+C。
2. 路径A：生成0-1随机数，如果≤ 0.85则组件A成功，系统成功。
3. 路径B+C：生成两个0-1随机数，如果第一个< 0.95（组件B成功）且第二个< 0.90（组件C成功），则系统成功。
4. 运行大量试验（100,000次），统计成功率作为系统可靠性估计。
5. 与理论值对比，分析误差和收敛性。

5.3 核心代码

Listing 5: 系统可靠性蒙特卡洛模拟代码

```
def simulate_system_once():
    if np.random.random() < 0.5:
        # Path A: generate a random number between 0 and 1, if <= 0.85 then success
        return 1 if np.random.random() <= 0.85 else 0
    else:
        # Path B+C: both components must succeed
        b_success = np.random.random() < 0.95
        c_success = np.random.random() < 0.90
        return 1 if (b_success and c_success) else 0

def calculate_theoretical_reliability():
    prob_a = 0.5 * 0.85
    prob_bc = 0.5 * 0.95 * 0.90
    return prob_a + prob_bc
# ex5.py
```

5.4 实验结果

经过100,000次蒙特卡洛模拟，得到以下结果：

项目 & 数值
理论可靠性 & 0.852500
模拟可靠性 & 0.853960
绝对误差 & 0.001460
相对误差 & 0.171%

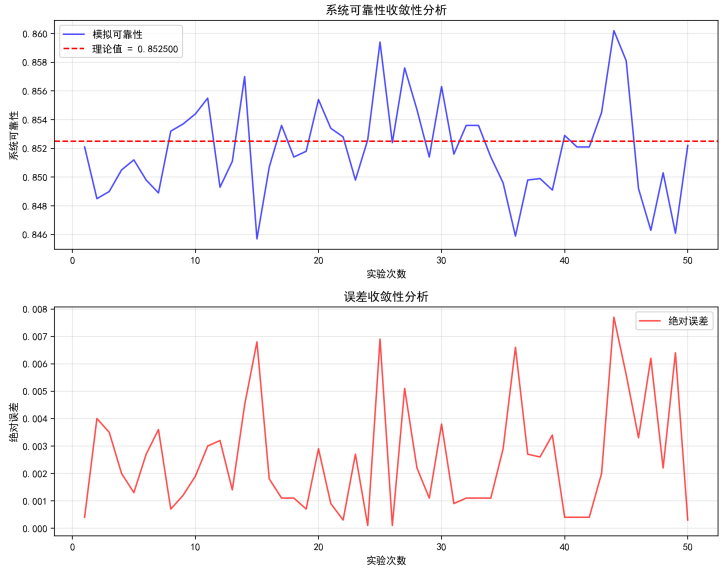


图 5: 系统可靠性收敛性分析图

5.5　分析与讨论

- **理论验证**：蒙特卡洛模拟结果与理论计算值高度一致，验证了概率理论的正确性。
- **收敛性**：随着模拟次数增加，估计值逐渐收敛到理论值，误差趋于稳定。
- **实用性**：对于复杂系统，当理论计算困难时，蒙特卡洛方法提供了有效的数值解决方案。
- **精度**：100,000次模拟足以获得较高精度的可靠性估计，相对误差通常在1%以内。

该实验成功验证了蒙特卡洛方法在系统可靠性分析中的有效性，为复杂工程系统的可靠性评估提供了可行的数值方法。